CSC3060 Project 4: Cache Simulator

Liangsen Wang 224040364@link.cuhk.edu.cn

Xiaoying Zeng 123090786@link.cuhk.edu.cn

1. Objectives

The goal of this project is to build and extend a simplified cache simulator so that you understand the most important ideas behind cache hierarchy design:

- how an address maps to a cache line, set, and tag
- how block offset, set index, and tag are derived from cache geometry
- how a cache access is classified as a hit or a miss
- what should happen on a hit and on a miss
- how a victim cache line is selected on replacement
- how write-back and write-allocate affect behavior
- how `L1` and `L2` caches form a hierarchy
- how replacement policies and prefetchers affect hit rate and average memory access time

This project has **three tasks**:

1. Complete the core access logic for a single-level `L1` cache.
2. Add an `L2` cache below `L1` and understand how a multi-level cache hierarchy works.
3. On a personalized trace distributed by the instructor, design suitable replacement policies and prefetchers to optimize hit rate and AMAT.

Team Policy

- You may work either individually or in a team of up to **two students**.
- If you work in a team, choose **one** teammate's student ID as the trace-generation seed.
- The same chosen student ID must be used consistently for:
  - generating the personalized trace
  - naming the submitted ZIP and PDF files
  - identifying the submission in the report
- Your report must clearly list:
  - both team members' names
  - both team members' student IDs
  - the chosen seed student ID used for trace generation

---

2. What You Are Given

The starter code contains the following files:

- `main.cpp`
  - Builds the memory hierarchy and replays the trace.
- `memory_hierarchy.cpp` / `memory_hierarchy.h`
  - Core logic for cache and memory accesses.
- `repl_policy.cpp` / `repl_policy.h`
  - Interfaces and scaffolding for replacement policies.
- `prefetcher.cpp` / `prefetcher.h`
  - Interfaces and scaffolding for prefetchers.
- `defs.h`
  - Basic data structures such as `CacheLine` and `CacheConfig`.
- `interfaces.h`
  - Abstract interfaces for memory objects, replacement policies, and prefetchers.
- `trace_sanity.txt`
  - A validation trace for Task 1 and Task 2.
- your personalized trace
  - Generated by the instructor from your student ID and used for Task 3.
  - For a two-student team, generate it using the chosen seed student ID of one teammate.

---

3. Simulator Input and Output

3.1 Trace Format

Each line in a trace file represents one memory access:

```
r 0x10000140
w 0x10000180
```

where:

- `r` means read
- `w` means write
- the second field is a hexadecimal address

3.2 Command Line Usage

Basic usage:

```
./cache_sim trace_sanity.txt 32 8 64 1 100
```

This runs a single-level hierarchy:

```
L1 -> Main Memory
```

The parameters mean:

```
./cache_sim <trace> <L1_KB> <assoc> <block> <L1_lat> <Mem_lat>
```

For example:

- `32`: L1 cache size is 32KB
- `8`: 8-way set associative
- `64`: block size is 64 bytes
- `1`: L1 hit latency is 1 cycle
- `100`: main memory latency is 100 cycles

The starter code also includes optional parameters:

```
./cache_sim <trace> 32 8 64 1 100 [L1_policy] [L1_prefetcher] [--enable-l2 [L2_policy]
[L2_prefetcher]]
```

Notes:

- `--enable-l2` explicitly turns on the Task 2 / Task 3 multi-level hierarchy.
- If `--enable-l2` is omitted, the simulator runs in Task 1 mode with only `L1`.
- In the starter code, the flag is parsed for you, but you still need to implement the actual `L2` creation and hookup in `main.cpp`.
- `L2_policy` defaults to `L1_policy` when `--enable-l2` is present.
- `L2_prefetcher` defaults to `None`.

Examples:

```
./cache_sim trace_sanity.txt 32 8 64 1 100 LRU None
./cache_sim my_trace.txt      32 8 64 1 100 BIP Stride --enable-l2 SRRIP None
```

---

4. Task 1: Single-Level L1 Cache

Background Knowledge

Before implementing Task 1, you should understand the following ideas:

- A memory address can be divided into:
  - **block offset** bits
  - **set index** bits
  - **tag** bits
- The number of **offset bits** depends on the cache line size.

- The number of **index bits** depends on the number of sets.
- The remaining upper bits form the **tag**.
- A hit occurs when a valid line in the selected set has the same tag.
- A miss occurs when no matching valid line exists.
- If a set is full, a replacement policy chooses the victim line.

What You Must Understand

For this task, you are expected to understand and correctly implement:

- how block offset is determined by cache line size
- how set index is determined by the number of sets
- how tag is derived from the remaining upper bits
- why changing cache size, associativity, number of sets, or cache line size changes address mapping
- how write-back and write-allocate affect cache state

Files You Must Modify

- `memory_hierarchy.cpp`
  - complete:
    - `get_index(...)`
    - `get_tag(...)`
    - `reconstruct_addr(...)`
    - `CacheLevel::access(...)`
- `repl_policy.cpp`
  - at minimum, implement `LRU`

TODOs

For Task 1, you must:

1. Implement address decomposition helpers.
2. Compute index and tag from each address.
3. Search the selected set for a hit.
4. On a hit:
   - update hit statistics
   - update replacement metadata
   - set dirty bit on writes
5. On a miss:
   - update miss statistics
   - search for an invalid line first
   - if necessary, select a victim

    - write back a dirty victim
    - fetch the requested line from the next level
    - install the new line
6. Make sure the implementation works for different cache geometries, not only one fixed configuration.

How To Do It

A good implementation order is:

1. Finish `get_index(...)`.
2. Finish `get_tag(...)`.
3. Finish `reconstruct_addr(...)`.
4. Implement hit detection in `access(...)`.
5. Implement miss handling in `access(...)`.
6. Implement `LRU` in `repl_policy.cpp`.

How To Test Task 1

You must test more than one cache geometry. During marking, your code may be tested under different values of:

- cache size
- associativity
- number of sets
- cache line size

At minimum, use the default target first and then try additional geometries:

```
make task1
./cache_sim trace_sanity.txt 16 4 64 1 100
./cache_sim trace_sanity.txt 32 4 32 1 100
./cache_sim trace_sanity.txt 64 8 128 1 100
```

Your code should still correctly:

- compute tag / index / offset
- detect hits and misses
- choose victims correctly
- report reasonable hit rate and AMAT

Result for reference

For the default `Makefile` target below (condition: only L1, LRU, no prefetch):

```
make task1
```

Expected output:

```
Constructed L1: 32KB, 8-way, 1cyc, [LRU + None]

=== Starting Simulation ===

=== Simulation Results ===
  [L1] Hit Rate: 21.43% (Access: 56, Miss: 44, WB: 2)
      Prefetches Issued: 0
  [Main Memory] Total Accesses: 46

Metrics:
  Total Instructions: 56
  Total Cycles:       4456
  AMAT:               79.57 cycles
```

Experimental Workflow for Task 1

1. Start with `L1 + LRU + None`.
2. Run `make task1` on `trace_sanity.txt` with the default geometry.
3. Change cache size and associativity.
4. Change cache line size.
5. Verify that the logic still works under all tested configurations.
6. Record representative outputs for your report.

---

5. Task 2: Add L2 and Understand Multi-Level Hierarchy

Background Knowledge

A multi-level cache hierarchy reduces average access latency by placing a larger, slower cache below a smaller, faster cache.

You should understand:

- what happens on an `L1` hit
- what happens on an `L1` miss followed by an `L2` hit
- what happens on an `L1` miss followed by an `L2` miss
- how dirty write-backs propagate to lower levels
- why `L2` can reduce traffic to main memory

Files You Must Modify

- `main.cpp`
  - create and connect `L2`
- `memory_hierarchy.cpp`
  - ensure your miss path works correctly in a recursive hierarchy
- `repl_policy.cpp`

    - if needed, continue improving replacement behavior

TODOs

For Task 2, you must:

1. Create an `L2` cache object and enable it only when `--enable-l2` is passed.
2. Connect the hierarchy as:

```
L1 -> L2 -> Main Memory
```

3. Print `L2` statistics.
4. Verify that some `L1` misses are served by `L2` rather than main memory.
5. Explain in your report how the hierarchy behaves.

How To Do It

A good implementation order is:

1. Keep your Task 1 `L1` logic unchanged.
2. Reuse the provided `--enable-l2` command-line option in `main.cpp`.
3. Instantiate `L2` only when the flag is present.
4. Change the next level of `L1` from `memory` to `l2` only in that mode.
5. Print both `L1` and `L2` statistics.
6. Compare behavior before and after connecting `L2`.

How To Test Task 2

Use `trace_sanity.txt` first through the default target, then test a few different cache geometries.

For example:

```
make task2
./cache_sim trace_sanity.txt 16 4 64 1 100 --enable-l2
```

Compare:

- `L1` hit rate
- `L2` hit rate
- number of main-memory accesses
- total cycles
- AMAT

Result for reference

For the default `Makefile` target below (condition: L1 + L2, LRU, no prefetch):

```
make task2
```

Expected output:

```
Constructed L2: 128KB, 8-way, 4cyc, [LRU + None]
Constructed L1: 32KB, 8-way, 1cyc, [LRU + None]

=== Starting Simulation ===

=== Simulation Results ===
  [L1] Hit Rate: 21.43% (Access: 56, Miss: 44, WB: 2)
      Prefetches Issued: 0
  [L2] Hit Rate: 50.00% (Access: 46, Miss: 23, WB: 0)
      Prefetches Issued: 0
  [Main Memory] Total Accesses: 23

Metrics:
  Total Instructions: 56
  Total Cycles:       2532
  AMAT:               45.21 cycles
```

Experimental Workflow for Task 2

1. Run `make task1`.
2. Run the same trace again with `make task2`.
3. Compare the same workload before and after connecting `L2`.
4. Compare memory traffic and AMAT.
5. Repeat with at least one more cache configuration.
6. Summarize how `L2` changes the access path and why it helps.

---

6. Task 3: Optimize Replacement Policy and Prefetcher

Background Knowledge

Different traces have different memory-access patterns. Some workloads are more sequential, some have stable strides, and some are affected strongly by cache pollution and replacement behavior.

You should understand:

- why `LRU`, `SRRIP`, and `BIP` behave differently
- why next-line prefetching helps on sequential accesses
- why stride prefetching helps on fixed-stride accesses
- why aggressive prefetching may hurt due to cache pollution

- why different cache levels may benefit from different policies

Files You Must Modify

- `repl_policy.cpp`
  - implement advanced policies such as `SRRIP` and `BIP`
- `prefetcher.cpp`
  - implement `NextLine` and `Stride`
  - optionally implement your own prefetcher
- `memory_hierarchy.cpp`
  - complete `install_prefetch(...)`
  - reuse the write-back helper when needed

TODOs

For Task 3, you must:

1. Implement at least the provided prefetchers and advanced replacement policies required by the starter.
2. Test combinations of replacement policy and prefetcher.
3. Analyze the access pattern of your personalized trace.
4. Choose a final design based on evidence from experiments.
5. You may implement your own prefetcher if you believe it is a better match for your trace.

How To Do It

A good implementation order is:

1. Finish `SRRIP` and `BIP`.
2. Finish `NextLine` and `Stride`.
3. Complete `install_prefetch(...)`.
4. Run several policy / prefetcher combinations.
5. Analyze why some combinations help and others hurt.
6. If useful, design your own prefetcher and justify it.

Trace Analysis Is Required

You should analyze your personalized trace before deciding on a design.

Before generating your personalized trace, compile the trace generator:

```
make trace_gen
```

This command builds:

```
trace_generator/workload_gen
```

If you prefer to compile it manually, you can also run:

```
g++ -std=c++11 -O2 -Wall -o trace_generator/workload_gen trace_generator/workload_gen.cpp
```

Generate your personalized trace with the provided trace generator:

```
./trace_generator/workload_gen student <student_id> > my_trace.txt
```

If you work in a two-student team, `<student_id>` should be the chosen seed student ID from one of the two teammates.
Use that same student ID consistently in your report and submission file names.

For example:

```
./trace_generator/workload_gen student 123456789 > my_trace.txt
```

Questions you should consider include:

- Is the trace mostly sequential?
- Does it contain a stable stride?
- Does it show short-term reuse?
- Is it scan-heavy?
- Is there evidence of cache pollution?
- Would a more conservative or more aggressive policy help?

A strong Task 3 solution is based on understanding the trace, not only on brute-force trial.

Starter Trace Analyzer

The starter code includes a basic helper script:

```
python3 trace_analyzer.py my_trace.txt
```

You may also tune the analysis granularity if needed:

```
python3 trace_analyzer.py my_trace.txt --block-size 64 --window-size 512 --top 10
```

It can summarize:

- read / write ratio
- unique block count
- common block strides
- frequently used blocks and `L1` sets
- simple per-window phase changes

You are encouraged to extend this script or write your own analysis tooling if you want deeper evidence for your report.

What The Trace Analyzer Looks At

The script first parses every memory access in the trace and converts each address into a **block address** using the cache block size.
It then computes several simple but useful statistics.

1. Basic access mix

The analyzer prints:

- total number of accesses
- number and percentage of reads
- number and percentage of writes
- number of unique blocks touched
- overall address range

This helps you answer questions such as:

- Is the workload read-heavy or write-heavy?
- Does it repeatedly touch a small footprint, or sweep across many distinct blocks?

2. Block-stride distribution

The analyzer compares consecutive block addresses and reports the most common stride values.
For example:

- stride `1` means the next access often goes to the next cache block
- stride `2` means every other block is common
- large positive strides may indicate regular jumps
- negative strides may indicate backward scans or loop resets

This section is useful for deciding whether prefetching is likely to help.
If one small stride dominates, `NextLine` or `Stride` prefetching is usually worth testing.

3. Hot blocks

The analyzer reports the most frequently accessed blocks.
If a small number of blocks appear very often, that usually indicates a **hot working set** or strong temporal locality.

This can help explain why:

- recency-based policies like `LRU` may work well
- some data remains useful even when the trace contains other noisy accesses

4. Hot L1 sets

The analyzer maps each block to an `L1` set using the cache geometry and reports the most frequently used sets.
This tells you whether accesses are spread out across the cache or concentrated into only a few sets.

If only a few sets dominate, that may indicate:

- conflict pressure
- set thrashing
- a case where replacement policy matters more than raw cache size

5. Per-window phase summary

The analyzer splits the trace into fixed-size windows and prints, for each window:

- number of reads
- number of writes
- number of unique blocks
- number of unique sets used in that window

This helps reveal whether the trace has multiple phases.
For example, one window might show strong reuse, while another looks like a streaming scan.
That kind of phase behavior is important when you explain why one policy helps in some regions but not others.

How To Use These Results In Your Report

A good report does not only paste analyzer output.
You should connect the statistics to your design choices.

For example:

- If the stride histogram is dominated by a small constant stride, explain why a prefetcher is a reasonable choice.
- If the trace touches many unique blocks with little repetition, explain why cache pollution may matter.
- If only a few sets are very hot, discuss possible conflict misses.
- If windows show different behavior, mention that the trace is phase-based rather than uniform.

In other words, your report should follow the pattern:

```
observation from trace analysis
-> hypothesis about cache behavior
-> policy / prefetcher choice
-> experimental result that supports or rejects the hypothesis
```

How To Test Task 3

In the commands below, `my_trace.txt` means the personalized trace you generated in the previous step using:

```
./trace_generator/workload_gen student <student_id> > my_trace.txt
```

You should compare multiple configurations. The default submission workflow is to encode your final Part 3 design in `Makefile` and let the marker run:

```
make task3
```

The provided `task3` target reads the following variables from `Makefile`:

- `TASK3_ASSOC`
- `TASK3_BLOCK`
- `TASK3_L1_POLICY`
- `TASK3_L1_PREFETCH`
- `TASK3_L2_POLICY`
- `TASK3_L2_PREFETCH`

For submission, you must edit these variables directly in `Makefile`.
Command-line temporary overrides are not allowed in the grading workflow.
The marker should be able to run your final Part 3 configuration using only:

```
make task3
```

For example, your final tuned values should be written into `Makefile` as fixed settings such as:

```
TASK3_ASSOC = 16
TASK3_BLOCK = 128
TASK3_L1_POLICY = SRRIP
TASK3_L1_PREFETCH = Stride
TASK3_L2_POLICY = LRU
TASK3_L2_PREFETCH = None
```

For development and analysis, you should still compare multiple configurations, for example:

```
./cache_sim my_trace.txt 32 8 64 1 100 LRU None
./cache_sim my_trace.txt 32 8 64 1 100 LRU NextLine
./cache_sim my_trace.txt 32 8 64 1 100 LRU Stride
./cache_sim my_trace.txt 32 8 64 1 100 SRRIP None
./cache_sim my_trace.txt 32 8 64 1 100 SRRIP NextLine
./cache_sim my_trace.txt 32 8 64 1 100 SRRIP Stride
./cache_sim my_trace.txt 32 8 64 1 100 BIP None
./cache_sim my_trace.txt 32 8 64 1 100 BIP NextLine
./cache_sim my_trace.txt 32 8 64 1 100 BIP Stride
```

If you connect `L2`, you may also test different policy / prefetcher combinations at different levels during development.

For example:

```
./cache_sim my_trace.txt 32 8 64 1 100 LRU None --enable-l2 SRRIP None
./cache_sim my_trace.txt 32 8 64 1 100 BIP Stride --enable-l2 SRRIP NextLine
```

Experimental Workflow for Task 3

1. Start by setting a simple baseline such as `LRU + None` in `Makefile`, then run `make task3`.
2. Compare `NextLine` and `Stride`.
3. Compare `LRU`, `SRRIP`, and `BIP`.
4. Test mixed combinations across levels if implemented.
5. Analyze the trace and explain the observed results.
6. Choose the best-performing design and justify it.
7. Try optional methods to lower your AMAT better than Best_AMAT.

Grading Rules for Task 3

The score for this part is determined solely by the **Student_AMAT** relative to the **Best_AMAT** and **Baseline_AMAT**, according to the following rules:

- **Best_AMAT** is the optimal achievable AMAT for the given trace (using the best combination of LRU/BIP/SRRIP and NextLine/Stride Prefetcher).
- **Baseline_AMAT** is the reference AMAT from a default configuration (L1 + L2 with no prefetching + LRU).

Scoring Criteria

Condition	Score
Student_AMAT < Best_AMAT	**50 points** (full score, better than reference)
Student_AMAT = Best_AMAT	**40 points**
Best_AMAT < Student_AMAT < Baseline_AMAT	Linearly interpolated between 0 and 40 points
Student_AMAT ≥ Baseline_AMAT	**0 points**

Linear Interpolation Formula

For `Best_AMAT < Student_AMAT < Baseline_AMAT`:

```
Score = 40 × (Baseline_AMAT - Student_AMAT) / (Baseline_AMAT - Best_AMAT)
```

Example Scoring Table (with Sample Values)

Assume:

- **Best_AMAT** = 2.0 cycles
- **Baseline_AMAT** = 20.0 cycles

Student_AMAT	Condition	Score
1.5	< Best_AMAT	50.0

Student_AMAT	Condition	Score
2.0 (reference Best_AMAT)	= Best_AMAT	40.0
5.0	Middle	33.33
10.0	Middle	22.22
15.0	Middle	11.11
20.0 (reference Baseline_AMAT)	≥ Baseline_AMAT	0.0
25.0	≥ Baseline_AMAT	0.0

> **Note:** The values above are illustrative. In actual grading, **Best_AMAT** and **Baseline_AMAT** will be determined per student trace.

---

7. Build and Run

Build

```
make
```

Clean

```
make clean
```

Example for Task 1

```
make task1
```

Example for Task 2

```
make task2
```

This command will work after you complete the Task 2 `L2` hookup in `main.cpp`.

Example for Task 3

```
make task3
```

In this setup, `task1`, `task2`, and `task3` are the unified default entry points for testing. `task1` and `task2` keep the default geometry for grading consistency, while `task3` allows you to tune `associativity`, `block size`, and policy choices for your personalized trace by editing `Makefile` before submission.

---

8. Output Metrics

The simulator prints several useful metrics:

- `Hit Rate`
  - fraction of accesses that hit in this cache level
- `Access`
  - total number of accesses to this level
- `Miss`
  - number of misses at this level
- `WB`
  - number of dirty write-backs to the next level
- `Prefetches Issued`
  - number of prefetches launched
- `Total Cycles`
  - total execution cycles in the simulation
- `AMAT`
  - Average Memory Access Time

In most cases, your main optimization goal is:

- **minimize** `AMAT`

while also understanding why changes in hit rate do or do not translate into better total performance.

---

9. Submission Requirements

Your submission **must** contain:
(Note: A 3-point penalty will be applied for incorrect file format or missing name/student ID in the report.)

If you work in a team of two, all submission file names must use the **chosen seed student ID** rather than the other teammate's ID.

1. **Source Code** in ZIP format
   - Submit your solution as a single ZIP archive named `StudentID_HW4.zip` (e.g., `123456789_HW4.zip`)
   - For a two-student team, `StudentID` here must be the chosen seed student ID used to generate `my_trace.txt`
   - The ZIP file must contain the **complete project directory** with the following structure:

```
project4/
├── ASSIGNMENT.md
├── defs.h
├── interfaces.h
```

```
├── main.cpp
├── Makefile
├── memory_hierarchy.cpp
├── memory_hierarchy.h
├── prefetcher.cpp
├── prefetcher.h
├── README.md
├── repl_policy.cpp
├── repl_policy.h
├── trace_sanity.txt
└── my_trace.txt
```

2. **Report** in PDF format
   - Submit your report in a PDF file named `StudentID_HW4.pdf` (e.g., `123456789_HW4.pdf`)
   - For a two-student team, `StudentID` here must be the chosen seed student ID used to generate `my_trace.txt`
   - the report is required and must contain all sections listed below

Required Report Contents

Your report **must** include the following sections:

1. **Student / Team Information**
   - for an individual submission: your name and student ID
   - for a two-student team: both names, both student IDs, and the chosen seed student ID
   - missing name / student ID / chosen seed information will result in a 3-point penalty
2. **Implementation Summary**
   - what you completed in Task 1, Task 2, and Task 3
3. **Address Mapping Explanation**
   - how you compute block offset, set index, and tag
   - how these change when cache geometry changes
4. **Task 1 Testing**
   - how you tested correctness under multiple cache configurations
   - representative results
5. **Task 2 Hierarchy Explanation**
   - how `L1`, `L2`, and memory interact
   - what changed after adding `L2`
6. **Task 3 Design Choices**
   - which replacement policies and prefetchers you implemented
   - whether you designed your own prefetcher
7. **Trace Analysis**
   - what access patterns you observed in your personalized trace
   - how those patterns influenced your design decisions

    - which student ID was used as the trace-generation seed
8. **Experimental Results**
    - tables comparing configurations
9. **Best Configuration and Discussion**
    - your best-performing design
    - why it performs well
    - where it may still fail
10. **External Resources and AI Usage**

- list any external resources you relied on, including websites, textbooks, friends, or LLMs
- if you use LLM tools such as ChatGPT, include the conversation link at the end of your report

A report that only lists final numbers is not sufficient.
You must explain the reasoning behind your design and results.
For team submissions, the report, trace seed, ZIP name, and PDF name must all use the same chosen seed student ID.

Late Policy

- The Project 4 deadline is **2026-04-13 23:59**.
- A penalty of 10 percent of the actual score will be applied for each day after the deadline.
- Submissions more than three days late will receive a score of zero.
- Please start this project early. If you encounter unexpected issues, email the Teaching Team to request an extension with a valid excuse and proof.

---

10. Grading Rubric

The project will be graded using the following components.

Task 1 Correctness — 30%

- correct address decomposition
- correct hit / miss handling
- correct victim selection behavior
- correct dirty write-back behavior
- correctness under different cache geometries

Task 2 Multi-Level Cache — 10%

- correct `L2` integration
- correct recursive access behavior
- correct `L2` statistics
- clear understanding of how multi-level caches work

Task 3 Optimization Design — 50%

- implementation of advanced replacement policies / prefetchers
- quality of experimental comparison
- effectiveness of the final design on the personalized trace
- optional originality beyond the provided baseline mechanisms

Report Quality — 10%

- completeness of required sections
- clarity of explanations
- quality of trace analysis
- quality of result presentation and discussion

---

11. Frequently Asked Questions

Q1. Do I need to implement every policy?

No, a reasonable minimum is usually:

- `LRU`
- one more advanced replacement policy
- `NextLine` or an appropriate prefetcher